

# LI.FI VM Audit Report



# Table of Contents

1. Executive Summary

4

Executive Summary

4

Risk Profile

4

2. Assumptions and Considerations

6

Assumptions & Considerations

6

Trust Assumptions

6

Security Assumptions

6

3. Severity Definitions

7

Impact

7

Likelihood

7

Severity Classification Matrix

7

4. Findings

9

L01: Arithmetic overflow in OP\_DIV\_UP operation when operand sum exceeds uint256 maximum

9

L02: packExplode shiftAmount calculation can underflow

10

L03: Insufficient guardrails on depositApproved may lead to accounting errors

11

5. Enhancement Opportunities

12

E01: Deploy ProxyFactory using CREATE3 for more robustness

12

E02: Unchecked dest parameters in packExplode/unpackExplode

13

E03: Missing validation on blueprintLength in unpackAbiEncode

14

E04: Unbounded return data

15

E05: Redundant pointer in \_encodeFlatCoreInto

16

E06: Unused \_encodeFlatCore function

17

E07: Use MAX\_ABI\_BP and PAX\_CDB\_BP constant in PackAbiEncode and (un)packCallDataBuild

18

E08: [Spec] MSB of srcReg and valueReg in the CALL command is not used

19

E09: Missing zero address check

20

E10: Remove unused variables

21

E11: Use smaller data types when possible 22

E12: Avoid using magic numbers 23

## About Us 24

About Adevar Labs 24

Audit Methodology 24

1. Program Context and Architecture Analysis 24

2. Threat Modeling 24

3. In-depth Manual Security Review 25

4. Detailed Fix Review and Validation 25

Confidentiality Notice 25

Legal Disclaimer 25

## Appendix A: Achieving Deterministic and Consistent Contract Deployment Across EVM Chains 27

LI.FI Virtual Machine 27

EVM options 27

Factory Contract 28

LI.FI ProxyFactory Contract 28

Unique address across all chains 28

# 1. Executive Summary

## Executive Summary

### About Li.Fi VM

The Li.Fi VM protocol is an on-chain virtual machine that builds and executes arbitrary call sequences to external contracts using an internal register system. It's designed to compose interactions in a single execution that will be invoked by the Li.Fi Diamond.

The system enables users to leverage an internal register system for both static and dynamic data types making it a complex but highly flexible environment to create execution flows. The VM operates on a highly gas-efficient custom DSL known as a "Blueprint" to construct calldata based on register values while also using a set of opcodes to provide additional functionality including **CALL**, **CALLDATA\_SURGERY**, **ABI\_ENCODE** and more. It also integrates with the ERC20 standard and allows users to deposit tokens through the **DEPOSIT\_APPROVED** opcode.

### Audit Summary

- Number of Findings: 3 total
- Critical: 0
- High: 0
- Medium: 0
- Low: 3
- Number of Enhancement Opportunities: 12

### Risk Profile

The following table summarizes the distribution of identified vulnerabilities by risk level:

| Risk Level | Count | Fixed | Acknowledged |
|------------|-------|-------|--------------|
| Critical   | 0     | 0     | 0            |
| High       | 0     | 0     | 0            |
| Medium     | 0     | 0     | 0            |
| Low        | 3     | 3     | 0            |

### Overall Security Posture

#### Before Fix Review

The Li.Fi team demonstrated a strong understanding of the inner workings of the EVM while ensuring extensive test coverage across all components of the VM. The Blueprint DSL also features high-quality documentation and examples for the various data types it can represent within the ABI encoding.



Our security review identified several low and informational level issues that addressed improving data validation across the inputs/outputs of the system, accounting errors, and arithmetic overflow/underflow.

## After Fix Review

All issues were resolved and the relevant enhancements to the project's use case were applied.

## Launch Recommendations

### Before Fix Review

We recommend addressing these issues before launch as they will limit the potential for user error when interacting with the VM. Additionally ensure any new functionality is added with a similar strategy in mind.

Consider reviewing the procedure for deployment to ensure consistent addressing for proxies over multiple chains (see Appendix A for details)

## Audit Scope

- **Repository:** <https://github.com/lifinance/Yggdrasil-VirtualMachine>
- **Commit Hash:** 3caed992b298763f6cabb3c6e680de10701dab32
- **Files/Modules in Scope:**
  - BlueprintEncoder.sol
  - CommandPacking.sol
  - DataModel.sol
  - DepositApproved.sol
  - InvariantChecker.sol
  - RegisterFile.sol
  - RegisterHelpers.sol
  - RPNArithmetic.sol
  - SurgeryOPS.sol
  - VirtualMachine.sol
  - VmConstants.sol
  - VmErrors.sol
  - VMLogLib.sol

## After Fix Review

The deployment procedure was reviewed and the approach suggested in Appendix A was implemented using the create3 deployer for user proxies.

- **After fix commit hash:** e4601773690d9c852f94f5119cca455c40bc521d

## 2. Assumptions and Considerations

---

### Assumptions & Considerations

#### Trust Assumptions

- In V0, LI.FI will be responsible for generating the calls and calling the Virtual Machine through the diamond
- External calls are performed on trusted targets only

#### Security Assumptions

- Any native or ERC20 tokens left in the Virtual Machine following a transaction should be considered lost. Users of the VM are responsible for ensuring their transactions are consuming any value deposited within the same transaction.
- Only external calls to functions that have a single return value are currently supported
- A surgery OP should only be performed on static registers
- In v0, the orchestration service that produces the calls also simulates them and makes sure that the flow does not revert

## 3. Severity Definitions

---

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

### Impact

Impact reflects the potential consequences of the issue—particularly on **project funds, user funds**, and the **availability or integrity** of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

### Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

- **High Likelihood:** The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood:** This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood:** Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

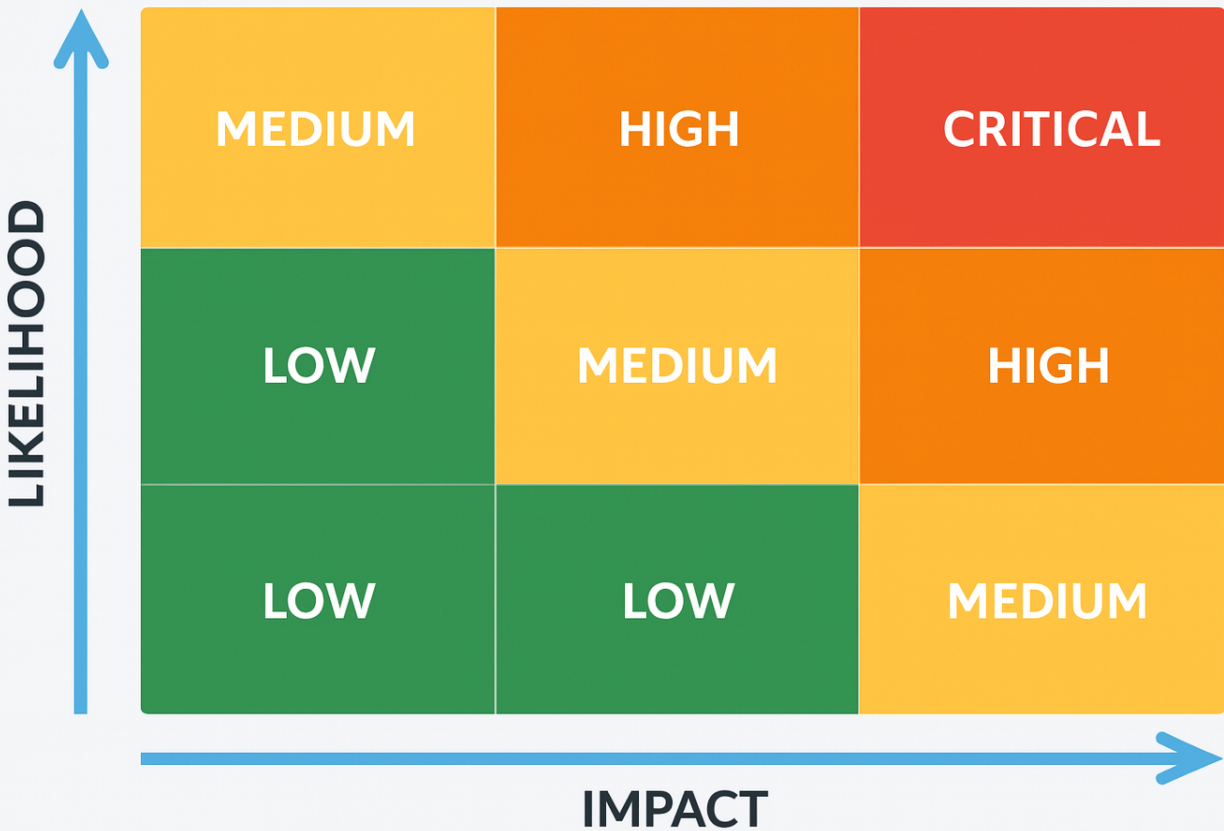
### Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical:** High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Moderate impact and/or discoverability
- **Low:** Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.





Severity Matrix

## 4. Findings

### L01: Arithmetic overflow in OP\_DIV\_UP operation when operand sum exceeds uint256 maximum

Status:

Resolved

Impact:

Low Medium High

Likelihood:

Low Medium High

Severity:

Low

**Location:** <https://github.com/lifinance/Yggdrasil-VirtualMachine/blob/3caed992b298763f6cabb3c6e680de10701dab32/src/RPNArithmetic.sol#L102>

>\_ RPNArithmetic.sol

SOLIDITY

```
100:      }
101:      // Integer division floors, so (a + b - 1) / b will equal ceil(a /
      b)
102:      c = a == 0 ? 0 : (a + b - 1) / b;
103:    } else {
104:      revert InvalidOpcode(op);
```

#### Description:

The current OP\_DIV\_UP implementation performs addition before division, which can cause arithmetic overflow when  $(a + b) > \text{max}(\text{uint256})$  even though the final result  $c < \text{max}(\text{uint256})$  would be valid.

The overflow occurs during the intermediate calculation step, causing the operation to revert unexpectedly.

#### Recommendation:

Implement the division with proper overflow handling.

Example of implementation:

- [Morpho](#)
- [Solady](#)

#### Developer Response:

Fix applied here: <https://github.com/lifinance/Yggdrasil-VirtualMachine/pull/27>.

## L02: packExplode shiftAmount calculation can underflow

Status:

Resolved

Impact:

Low Medium High

Likelihood:

Low Medium High

Severity:

Low

**Location:** <https://github.com/lifinance/Yggdrasil-VirtualMachine/blob/3caed992b298763f6cabb3c6e680de10701dab32/src/CommandPacking.sol#L186>

>\_ CommandPacking.sol

SOLIDITY

```
184:         let elementPtr := add(add(destRegs, 0x20), mul(i, 0x20))
185:         let regVal := and(mload(elementPtr), 0xFF)
186:         let shiftAmount := sub(232, mul(i, 8))
187:         packed := or(packed, shl(shiftAmount, regVal))
188:     }
```

### Description:

The **packExplode** function contains a potential arithmetic underflow in the **shiftAmount** calculation.

**sub(232, mul(i, 8))** can underflow when **i \* 8** exceeds 232, which occurs when **i >= 29**. While the function documentation states a maximum of 26 destination registers, there is no explicit validation to enforce this limit.

This will cause **packed** to be equal to **0** since a shift greater than 255 causes the result to be zero.

### Recommendation:

Prevent potential runtime errors, improve code reliability, and ensure the function behaves as documented:

```
function packExplode(
    uint8 sourceReg,
    uint8 destCount,
    uint8[] memory destRegs
)
    internal
    pure
    returns (bytes32 packed)
{
    if (destCount > 26) {
        revert VmErrors.TooManyDestinationRegisters();
    }
    // ... rest of function remains unchanged
}
```

SOLIDITY

### Developer Response:

Fixed here: <https://github.com/lifinance/Yggdrasil-VirtualMachine/pull/34>.

## L03: Insufficient guardrails on depositApproved may lead to accounting errors

Status:

Resolved

Impact:



Low

Medium

High

Likelihood:



Low

Medium

High

Severity:

Low

**Location:** <https://github.com/lifinance/Yggdrasil-VirtualMachine/blob/3caed992b298763f6cabb3c6e680de10701dab32/src/DepositApproved.sol#L59>

> \_DepositApproved.sol

SOLIDITY

```
57:         mstore(add(ptr, 0x04), address())
58:         ok := staticcall(gas(), token, ptr, 0x24, ptr, 0x20)
59:         if and(ok, eq returndatasize(), 0x20)) { pre := mload(ptr) }
60:     }
61:
```

### Description:

The **depositApproved** function is making low level assembly calls to an ERC20 contract for the **allowance**, **balanceOf**, and **transfer** functions. While the return status is checked for these calls, the function doesn't revert even if the calls themselves reverted. Consider the **balanceOf** call for getting the user's token balance before the **safeTransferFrom**.

```
// FOT support: get current balance
mstore(ptr, 0x70a0823100000000000000000000000000000000000000000000000000000000)
mstore(add(ptr, 0x04), address())
ok := staticcall(gas(), token, ptr, 0x24, ptr, 0x20)
if and(ok, eq returndatasize(), 0x20)) { pre := mload(ptr) }
```

If the static call fails, **pre** will maintain its default value of **0**, but the function execution will continue. The final return will be **post - pre = post - 0 = post** which will overinflate the actual token balance received. Subsequent operations that will depend on this amount for correct accounting will not be correct.

### Recommendation:

For every low level call made in the **depositApproved** function, if the call fails, the whole function should revert to prevent any further issues.

### Developer Response:

Fixed here: <https://github.com/lifinance/Yggdrasil-VirtualMachine/pull/36>



## 5. Enhancement Opportunities

### E01: Deploy ProxyFactory using CREATE3 for more robustness

**Location:** <https://github.com/lifinance/Yggdrasil-VirtualMachine/blob/3caed992b298763f6cabb3c6e680de10701dab32/src/proxy/ProxyFactory.sol#L48-L48>

> \_ ProxyFactory.sol

SOLIDITY

```
46:
47:      // Calculate salt based on user's address for deterministic deployment
48:      // This ensures same user always gets same proxy address on any chain
49:      bytes32 salt = keccak256(abi.encodePacked(user));
50:
```

#### Description:

The current deployment script uses the [Foundry deterministic deployment method](#). This ensures that the address of the contract will remain consistent on every chain.

However, this property only holds if the **ProxyFactory** contract's bytecode is fixed over time, which might be compromised in the future if:

- the init codehash is modified, which include: code modification (space/comments included), constructor
- the compiler configuration is modified, which includes: compiler version, EVM version, optimization steps, ...

[CREATE3](#) is a new deployment method developed to address these specific issues.

#### Potential Benefit:

Ensures long-term address stability across all supported chains, independent of contract bytecode modifications

#### Recommendation:

Use CREATE3 to deploy the **ProxyFactory** contract.

Non exhaustive list of tools allowing to achieve such goal:

- [Xdeployer](#)
- [CreateX-forge](#)
- [SKYBIT-Keyless-Deployment](#)

#### Developer Response:

Changed applied here: <https://github.com/lifinance/Yggdrasil-VirtualMachine/pull/25>.

Deployment scripts now use createXScript library, and ProxyFactory uses create3 deployer for user proxies, both using the CREATEX\_ADDRESS as the deployer.

## E02: Unchecked dest parameters in packExplode/unpackExplode

**Location:** <https://github.com/lifinance/Yggdrasil-VirtualMachine/blob/3caed992b298763f6cabb3c6e680de10701dab32/src/CommandPacking.sol#L166-L168>

> **CommandPacking.sol**

**SOLIDITY**

```
164:      * @return packed The packed 32-byte command.
165:      */
166:      function packExplode(
167:          uint8 sourceReg,
168:          uint8 destCount,
169:          uint8[] memory destRegs
170:      )
```

### Description:

There are multiple enhancements to be applied within these functions:

#### **packExplode:**

- Missing **destCount** bounds check between **0** and **26**
- Missing **destCount == destRegs.length** check

#### **unpackExplode:**

- Missing **destCount** bounds check between **0** and **26**

### Potential Benefit:

Validating these parameters will prevent misinterpreting padding bytes and reading out-of-bounds memory, preventing downstream memory issues.

### Recommendation:

To both functions add:

```
if (uint256(destCount) == 0 || uint256(destCount) > 26) {
    revert VmErrors.ExplodeDestCountOutOfBounds(destCount);
}
```

To **packExplode** add:

```
if (destRegs.length < uint256(destCount)) {
    revert VmErrors.ExplodeDestRegsTooShort(destRegs.length, destCount);
}
```

### Developer Response:

Fixes applied here: <https://github.com/lifinance/Yggdrasil-VirtualMachine/pull/34>.

## E03: Missing validation on blueprintLength in unpackAbiEncode

**Location:** <https://github.com/lifinance/Yggdrasil-VirtualMachine/blob/3caed992b298763f6cabb3c6e680de10701dab32/src/CommandPacking.sol#L455>

> **CommandPacking.sol**

**SOLIDITY**

```
453:
454:         // Extract blueprint length from byte 1.
455:         uint8 blueprintLength = uint8(uint256(packed >> 240));
456:
457:         // Create the blueprint bytes array.
```

### Description:

The blueprint length under the internal abi specifications requires a maximum length of 27 bytes. This requirement is enforced in **packAbiEncode** but not in **unpackAbiEncode** which may erroneously include padding bytes as part of the blueprint.

### Potential Benefit:

Helps prevent downstream parsing errors from the unvalidated length.

### Recommendation:

Add the missing validation to **unpackAbiEncode**

```
if (blueprint.length > 27) {
    revert VmErrors.BlueprintTooLarge();
}
```

### Developer Response:

Fix applied here: <https://github.com/lifinance/Yggdrasil-VirtualMachine/pull/26>.

## E04: Unbounded return data

**Location:** <https://github.com/lifinance/Yggdrasil-VirtualMachine/blob/3caed992b298763f6cabb3c6e680de10701dab32/src/VirtualMachine.sol#L77>

```
>_ VirtualMachine.sol SOLIDITY
75:         }
76:
77:         (bool success, bytes memory ret) = _performCall(c.target, CallType(c.c
allType), callData, value);
78:         if (!success) {
79:             // Bubble up the exact revert data from the failed call
```

### Description:

Op.CALL allows users to call arbitrary external contracts and currently doesn't perform any output data sanitization. This may pose a problem in the future where an external contract has the means of reverting transactions, for example with [this](#) attack vector.

### Potential Benefit:

Ensures the external contract is not behaving in a malicious way by returning unexpected data

### Recommendation:

Perform output data length sanitization using the ExcessivelySafeCall library

### Developer Response:

We acknowledge this potential issue and would like to proceed without it.

## E05: Redundant pointer in `_encodeFlatCoreInto`

**Location:** <https://github.com/lifinance/Yggdrasil-VirtualMachine/blob/3caed992b298763f6cabb3c6e680de10701dab32/src/BlueprintEncoder.sol#L543-L548>

> `_BlueprintEncoder.sol`

SOLIDITY

```
541:         if (dataLen < 32) revert BadDynamicFormat(idx);
542:
543:         // Calculate the absolute memory address for the tail data.
544:         uint256 absTail = headBase + tailPtr;
545:         // The pointer value is relative to the start of the ABI data block.
546:         uint256 relPtr = absTail - headBase;
547:         // Write the pointer to the head section.
548:         _writeUint(out, headBase + headPtr, relPtr);
549:         // Copy the entire dynamic blob (length-prefixed) to the tail section.
550:         uint256 written = _copyDyn(out, absTail, reg);
```

### Description:

There is a pointer named **relPtr** which contains the same value as **tailPtr**.

### Potential Benefit:

Better code health and auditability

### Recommendation:

Replace

```
// The pointer value is relative to the start of the ABI data block.
uint256 relPtr = absTail - headBase;
// Write the pointer to the head section.
_writeUint(out, headBase + headPtr, relPtr);
```

with

```
// Write the pointer to the head section.
_writeUint(out, headBase + headPtr, tailPtr);
```

### Developer Response:

Fixed here: <https://github.com/lifinance/Yggdrasil-VirtualMachine/pull/28>



## E06: Unused `_encodeFlatCore` function

**Location:** <https://github.com/lifinance/Yggdrasil-VirtualMachine/blob/3caed992b298763f6cabb3c6e680de10701dab32/src/BlueprintEncoder.sol#L562>

> `_BlueprintEncoder.sol`

SOLIDITY

```
560:
561:     /// @dev Helper to allocate and encode a flat blueprint (without selector).
562:     function _encodeFlatCore(
563:         bytes calldata bp,
564:         bytes[] memory regs,
```

### Description:

There is a defined `_encodeFlatCore` function but it is never used.

### Potential Benefit:

Removing the unused function will result in reduced contract deployment costs and better code health.

### Recommendation:

Remove the `_encodeFlatCore` function

### Developer Response:

Fixed here: <https://github.com/lifinance/Yggdrasil-VirtualMachine/pull/29>

## E07: Use MAX\_ABI\_BP and PAX\_CDB\_BP constant in PackAbiEncode and (un)packCallDataBuild

**Location:** <https://github.com/lifinance/Yggdrasil-VirtualMachine/blob/3caed992b298763f6cabb3c6e680de10701dab32/src/VmConstants.sol#L15-L17>

> **VmConstants.sol**

**SOLIDITY**

```
13:      uint8 public constant MAX_SURGERIES = 6;
14:
15:      // Blueprint size limits
16:      uint8 public constant MAX_CDB_BP = 22; // CALLDATA_BUILD blueprint max size
17:      uint8 public constant MAX_ABI_BP = 27; // ABI_ENCODE blueprint max size
18:
19:      // Blueprint container tokens (from BlueprintEncoder)
```

### Description:

In the current state both constant are unused, e.g:

```
function packAbiEncode(uint8 destReg, bytes memory blueprint) internal pure returns (bytes32 p
acked) {
    if (blueprint.length > 27) {
        revert VmErrors.BlueprintTooLarge();
    }
}
```

Enhancement locations:

- src/CommandPacking.sol:71
- src/CommandPacking.sol:138
- src/CommandPacking.sol:422

### Potential Benefit:

Improve code readability

### Recommendation:

Use implemented constants or remove them.

### Developer Response:

Fixed here: <https://github.com/lifinance/Yggdrasil-VirtualMachine/pull/33>

## E08: [Spec] MSB of srcReg and valueReg in the CALL command is not used

**Location:** <https://github.com/lifinance/Yggdrasil-VirtualMachine/blob/3caed992b298763f6cabb3c6e680de10701dab32/docs/isa.md?plain=1#L47-L48>

| >_isa.md |       |                 |    | MARKDOWN                                                                           |
|----------|-------|-----------------|----|------------------------------------------------------------------------------------|
| 45:      | 1-20  | Target Address  | 20 | External contract address                                                          |
| 46:      | 21    | Destination Reg | 1  | Register index for storing the call's return data; high bit indicates dynamic mode |
| 47:      | 22    | Source Reg      | 1  | Register index holding the pre-built calldata; high bit indicates dynamic mode     |
| 48:      | 23    | Value Reg       | 1  | Register index holding the call value; high bit indicates dynamic mode             |
| 49:      | 24-31 | Padding         | 8  | Reserved (must be zero)                                                            |
| 50:      |       |                 |    |                                                                                    |

**Description:**

The ISA specification indicates that: **high bit indicates dynamic mode** for both the source register and the value register of the **CALL** command. However, this bit is not relevant and does not need to be checked.

**Potential Benefit:**

Improve code maintainability and auditability.

**Recommendation:**

In order to improve maintainability we recommend to update the specification and remove this text from these 2 registers because future developers or auditors might be confused as to why the checks of the MSB are missing from the code.

**Developer Response:**

Fixed here: <https://github.com/lifinance/Yggdrasil-VirtualMachine/pull/32>

## E09: Missing zero address check

**Location:** <https://github.com/lifinance/Yggdrasil-VirtualMachine/blob/3caed992b298763f6cabb3c6e680de10701dab32/src/VirtualMachine.sol#L77>

> **VirtualMachine.sol**

**SOLIDITY**

```
75:         }  
76:  
77:         (bool success, bytes memory ret) = _performCall(c.target, CallType(c.c  
allType), callData, value);  
78:         if (!success) {  
79:             // Bubble up the exact revert data from the failed call
```

### Description:

As part of the VMCommand calldata, a user must specify a target address for the CALL. Currently, there is no check for the target being the zero address.

### Potential Benefit:

Making a call to the zero address is often a sign of a malformed user request and shouldn't be allowed. This check will reduce the number of such calls executed.

### Recommendation:

Ensure that the target address is not the zero address. For example, add the following in `_run`

```
if (c.target == address(0)) revert InvalidTarget();
```

### Developer Response:

Disallowing calls to the zero address prevents some use cases e.g. burning tokens via a zero address transfer, which we may want the VM to support

## E10: Remove unused variables

**Location:** <https://github.com/lifinance/Yggdrasil-VirtualMachine/blob/3caed992b298763f6cabb3c6e680de10701dab32/src/BlueprintEncoder.sol#L456>

> **BlueprintEncoder.sol**

**SOLIDITY**

```
454:     {  
455:         uint256 len = bp.length;  
456:         uint256 regCnt = regs.length;  
457:         for (uint256 i; i < len;) {  
458:             uint8 tok = uint8(bp[i]);
```

### Description:

The following variables are unused in the **BlueprintEncoder.sol** file:

```
456 |         uint256 regCnt = regs.length;  
519 |         uint256 regCnt = regs.length;  
848 |         uint256 regCount = registers.length;
```

### Potential Benefit:

Cleaner code

### Recommendation:

Remove unused code

### Developer Response:

Fixed here: <https://github.com/lifinance/Yggdrasil-VirtualMachine/pull/30>

## E11: Use smaller data types when possible

**Location:** <https://github.com/lifinance/Yggdrasil-VirtualMachine/blob/3caed992b298763f6cabb3c6e680de10701dab32/src/BlueprintEncoder.sol#L237>

> **BlueprintEncoder.sol**

**SOLIDITY**

```
235:
236:     // A word in the EVM is 32 bytes.
237:     uint256 internal constant LEN_WORD = 32;
238:
239:     // --- Bit-packing constants for `tmpDynHeads` and dynamic array metadata ---
```

### Description:

The **LEN\_WORD** and **SHIFT\_CNT** constants are defined as **uint** which occupy 32 bytes each. However, the values of these are well below the **uint8** maximum value of **256**.

### Potential Benefit:

Gas savings due to storage packing

### Recommendation:

Modify the **uint** data type to **uint8** for constants that fit within this storage capacity

### Developer Response:

Fixed here: <https://github.com/lifinance/Yggdrasil-VirtualMachine/pull/31>



## E12: Avoid using magic numbers

**Location:** <https://github.com/lifinance/Yggdrasil-VirtualMachine/blob/3caed992b298763f6cabb3c6e680de10701dab32/src/BlueprintEncoder.sol#L237>

> \_BlueprintEncoder.sol

SOLIDITY

```
235:
236:     // A word in the EVM is 32 bytes.
237:     uint256 internal constant LEN_WORD = 32;
238:
239:     // --- Bit-packing constants for `tmpDynHeads` and dynamic array metadata ---
```

### Description:

Magic numbers are numerical values placed in code without context and are used frequently throughout the BlueprintEncoder for values **32** and **4**. The LEN\_WORD constant is defined but not used sufficiently.

### Potential Benefit:

Removing magic numbers can lead to multiple benefits including better code clarity and reducing potential errors.

### Recommendation:

Use constants instead of magic numbers

```
//Examples
uint8 internal constant SELECTOR_SIZE = 4;
uint8 internal constant LEN_WORD = 32; //included but only used once
```

### Developer Response:

Fixed here: <https://github.com/lifinance/Yggdrasil-VirtualMachine/pull/42>

# About Us

---

## About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over \$500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

## Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

### 1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

### 2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification

- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

### 3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

### 4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

### Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

### Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.

## Appendix A: Achieving Deterministic and Consistent Contract Deployment Across EVM Chains

LI.FI's goal is to deploy every proxy to a unique and same address on every supported EVM chain. This problem is not new and has been addressed in many different ways over time. While the EVM allows for deterministic addresses through the **CREATE** and **CREATE2**, numerous issues can arise, which if not handled properly, can lead to situations where it is not possible anymore to achieve the desired outcome.

This appendix will explore the various techniques that can help us accomplish this goal, and why the **CREATE3** approach is the most robust one.

### LI.FI Virtual Machine

The LI.FI Virtual Machine is a contract that can perform a series of predetermined actions, such as deposits, external calls, logging, and more.

This Virtual Machine is designed to be used through a proxy, allowing for upgrades or enabling different entities to deploy their own instances with themselves as the owners.

The proxy is deployed through the **ProxyFactory::deployProxy** external function which uses **CREATE2** ([link](#)):

```
function deployProxy(address user) external returns (address) {  
    // Ensure user address is valid  
    if (user == address(0)) revert VmErrors.InvalidUserAddress();  
  
    // Check if proxy already exists by checking if there's already code at the determinis  
tic address  
    address proxyAddress = this.predictProxyAddress(user);  
    if (proxyAddress.code.length != 0) revert VmErrors.ProxyAlreadyDeployed();  
  
    // Calculate salt based on user's address for deterministic deployment  
    // This ensures same user always gets same proxy address on any chain  
    bytes32 salt = keccak256(abi.encodePacked(user));  
  
    // Deploy a new proxy with CREATE2 and the specified user as the owner  
    MinimalProxy proxy = new MinimalProxy{ salt: salt }(user, vmContract);  
  
    // Emit event  
    emit ProxyDeployed(user, address(proxy));  
  
    return address(proxy);  
}
```

SOLIDITY

### EVM options

To deploy a contract, the EVM offers two opcodes. The initial one is **CREATE** and can be triggered in one of two ways:

1. by an EOA when it sends a transaction with **to = address(0)** and **data = deployCode**.
2. by a contract using the **new Contract(argument1, ...)** syntax in Solidity, or **create(value, offset, size)** in assembly.

This contract creation method derives the future address as

`keccak256(rlp([sender_address, sender_nonce]))[12:]` where:

- **sender\_address** is the EOA address if triggered with the first method, or the contract address if the second method is used.
- **sender\_nonce** is a counter incremented on every transaction, even the transaction reverts.

**CREATE2** can only be triggered by a contract. This makes it easier to know the deployed address in advance, as it no longer depends on the nonce. In Solidity, this can be used with the **new Contract{salt: salt}(argument1, ...)** syntax or **create2(value, offset, size, salt)** in assembly. The derivation of the address uses the following calculation:

```
initialisation_code = memory[offset:offset+size]
address = keccak256(0xff + sender_address + salt + keccak256(initialisation_code))[12:]
```

**SOLIDITY**

In this case, **sender\_address** will always be the contract address, as the opcode is initiated by the contract.

## Factory Contract

In many situations, it is necessary for a protocol to deploy new contracts dynamically (e.g., deploying a new token pair). To simplify this process for users, they deploy a factory contract that uses either **CREATE** or **CREATE2** to create these new contracts.

To guarantee the uniqueness of the new pair in our example, we must use the **CREATE2** opcode. The token addresses could either be included in the salt or **initialisation\_code**. If we were using **CREATE** instead, a new address would be generated for each call, as address derivation depends on the nonce.

## LI.FI ProxyFactory Contract

In the context of LI.FI VM, we want to achieve two outcomes:

- 1) deploy a unique proxy tied to user
- 2) ensure that the proxy address remains the same across all chains

In order to achieve (1), we can use **CREATE2** with the user as a parameter or a salt.

To address (2), we need:

- to make sure that the **sender\_address** (the LI.FI Factory contract here) is the same for every chain, which means **ProxyFactory** must be deployed correctly.
- but also to make sure the **vmAddress** constructor parameter is the same across all chains, as it is part of the **initialisation\_code**, which means the VM might also need to be deployed using a similar method. An alternative would be to remove the **vmAddress** from constructor, and feed it through an initializer function.

## Unique address across all chains

### CREATE

The initial, straightforward method is to use a new address to deploy the **ProxyFactory** with the **CREATE** function, making sure that the nonce is identical for all deployments.

As a reminder, the contract's address is determined by

**keccak256(rlp([sender\_address, sender\_nonce]))[12:].**

Getting the same **sender\_address** is easily achievable, as a same private key will lead to the same public addresses on every EVM chain.

Managing the **sender\_nonce** appears straightforward, but there are some risks to consider. For instance, if a transaction reverts due to insufficient gas, the nonce will still increase. Additionally, there may be chain-specific issues like the one encountered with Base.

The LI.FI implementation follows a multistep deployment procedure. Specifically, the **ProxyFactory** is deployed in a separate transaction.

## CREATE2

The second approach consist in deploying the **ProxyFactory** contract through a **CREATE2** deployer. The **CREATE2** deployers are contract deployed using the first described method on multiple chains, alleviating the burden of managing this to other developers.

Thus, **CREATE2** deployers have the same address on every supported chains, and provide public functions to deploy arbitrary contracts.

Foundry implemented such feature, which is used by the current deployment script.

However, this property only holds if the **ProxyFactory** contract's bytecode is fixed over time, which might be compromised in the future if:

- the init codehash is modified, which include: code modification (space/comments included), constructor
- the compiler configuration is modified, which includes: compiler version, EVM version, optimization steps, etc

The second downside is that you might be dependent on the maintainer of the **CREATE2** factories to deploy to a currently unsupported chain. Although, there existing solutions that allows you to deploy those factories to new chains by yourself:

- Arachnid deterministic deployment proxy which is the one used by default in Foundry
- createX

If you wish to use a specific **CREATE2** deployer using foundry, you can add the **create2\_deployer** key in foundry configuration file

## CREATE3

To get around the fixed bytecode limitation of **CREATE2**, a more robust technique has been developed known as **CREATE3** (link). It accomplishes this by initially deploying a proxy using **CREATE2**, which subsequently deploys the desired contract using **CREATE**.

Similar to **CREATE2** deployers, this technique requires a consistent **CREATE3** deployer address across different blockchains.

The deployer first deploys a first proxy using **CREATE2**, which allows for a deterministic address based on a salt, the first proxy bytecode being hardcoded into the deployer.

Then, the contract to deploy is sent to the newly deployed proxy, which creates it using **CREATE**. This ensures that the nonce is always zero, while also allowing the resulting address to be independent of the target contract bytecode.

To prevent front-running attacks, it is essential that the **CREATE3** deployer incorporate **msg.sender** into the salt. Failing to do so allows a malicious actor to copy your deployment salt and deploy a separate contract at the same address.

Foundry does not currently support **CREATE3** deployment. A package ([createX-forge](#)) has been developed to address this specific use case. A better alternative might be the [SKYBIT Keyless Deployment tool](#), which also allows you to deploy your own **CREATE3** deployers. Their README document provides detailed information on any situations or edge cases you might encounter, so it is highly recommended that you read it before proceeding.